Do Agents Need an LLM to Remember? An Evaluation of LLM-Free Memory Ingestion

Accuracy parity with LLM-based memory systems, at three to four orders of magnitude lower write cost

Kristian Baer
Northtek Labs (FrostByte Digital LLC), Anchorage, Alaska, USA
info@northtek.io · Code: `github.com/NORTHTEKDevs/genome`
August 2026

Abstract. Most memory systems for AI agents, including Mem0, call a large language model on every incoming message to decide what to remember. That call is the dominant cost of the write path, it makes the stored record non-reproducible, and it ties ingestion to a network. This paper asks whether the call is necessary. I evaluate GENOME, an open-source memory layer whose default write path is a single local embedding: no LLM, no API, no network. Under a single controlled harness on two public benchmarks, LLM-free ingestion reaches answer-accuracy parity with Mem0: on LoCoMo, 0.851 versus 0.855 (paired McNemar $p = 0.747$, $n = 1{,}540$); on LongMemEval, GENOME leads nominally in two independent runs but never significantly ($p = 0.19$ at $n = 90$, $p = 0.14$ at $n = 205$). The measured write cost is 0 LLM calls per message versus Mem0's 1.00, a factor of roughly 8,900 on the measured run and 837 to 8,433 projected at deployment scale. In its default local-encoder configuration the write path completes in about 10 ms with all network sockets blocked, a property verified separately from the accuracy harness, which holds a single shared encoder across all systems for fairness. When conversation history overflows the context window, retrieval beats the stronger of two truncated full-context baselines by +0.409 accuracy using 80 times less context. An optional bi-temporal layer answers point-in-time questions ("what was true at time T?") at 0.870 versus Mem0's 0.676 ($p = 0.0007$). I also report the negative results: hybrid BM25-plus-dense retrieval underperformed plain dense, and summarizing old memories was statistically indistinguishable from deleting them at equal token budget. Everything regenerates from scripts in the public repository.

1 Introduction

An agent that talks to a user for months cannot keep the whole conversation in its context window, so it needs a memory layer: something that decides what to keep and returns the relevant pieces later. The current generation of memory systems makes an expensive assumption about the first half of that job. Mem0 [1], the most visible open-source system in this space, sends every incoming message to an LLM which extracts the facts worth keeping. MemGPT [2] similarly places an LLM in charge of paging information between context and storage. The assumption is that deciding what to remember requires intelligence at write time.

That assumption has three costs. The obvious one is money: one LLM call per message, on every message, forever. The second is reproducibility. An LLM extraction step is non-deterministic, so the same conversation ingested twice can produce different memory stores, which means the record cannot be re-derived or audited after the fact. The third is connectivity: a write path that needs an LLM API cannot run on an air-gapped machine.

The alternative is almost embarrassingly simple: embed the message with a local model and store it. Push all the intelligence to read time, where a query is available to focus it. If that works, the three costs disappear at once, because a local embedding is cheap, deterministic, and offline. The question is whether it works, meaning whether answer accuracy survives the removal of the LLM from the write path.

This paper measures that question. I built GENOME, an Apache-2.0 memory layer whose default ingestion is exactly the simple thing above, and evaluated it against Mem0 and against full-context baselines under a single harness that holds the responder model, the judge, the embedder, and the retrieval depth constant, so the memory layer is the only variable. The evaluation covers the LoCoMo long-conversation benchmark [3], the harder LongMemEval benchmark [4], a context-overflow stress test, a measured ingestion-cost comparison, and a bi-temporal extension for point-in-time queries. Wins, ties, and failures are reported with equal prominence, and every number in this paper regenerates from a script in the repository.

The headline result is a parity, and I want to be precise about that up front: on answer accuracy, GENOME does not beat Mem0, and this paper does not claim it does. GENOME ties Mem0 on both benchmarks while writing for roughly three to four orders of magnitude less money, roughly 8.6 times lower write latency as measured, and, in its default configuration, zero network dependence. The claim is that the LLM in the write path buys no measured accuracy, which makes it a cost without a benefit for the workloads tested here.

2 Related work

Retrieval-augmented generation [5] established the pattern of fetching relevant text into the context at query time; agent memory systems specialize it for conversational state. Mem0 [1] ingests through LLM fact-extraction and reports strong LoCoMo results; its authors' evaluation methodology, including the judge prompt, is the one I adopt verbatim for fairness. MemGPT [2] treats the LLM as an operating system that manages its own memory hierarchy. On the evaluation side, LoCoMo [3] provides very long multi-session conversations with annotated evidence turns, and LongMemEval [4] provides a harder retrieval setting, averaging 51 sessions per question with distractors. GENOME's embedder is Sentence-BERT-style [6], defaulting to a local MiniLM model. What distinguishes this work is not a new retrieval algorithm; it is the controlled measurement of whether write-time LLM intelligence earns its cost, plus the negative results from ablations that the field's marketing tends to skip.

3 System

Write path. A message arrives, is embedded by a local sentence-transformer (default `all-MiniLM-L6-v2`, about 90 MB, downloaded once), and is written with its vector and metadata to local storage (SQLite by default, Postgres with pgvector for larger deployments). Nothing else happens. There is no LLM call, no network traffic, and no non-determinism: ingesting the same conversation twice produces the same store. The write completes in roughly 10 ms on commodity hardware, and I verify the offline property directly by blocking all network sockets during a write run and observing that writes still succeed (`benchmarks/local_writepath.py`).

Read path. Retrieval is exact brute-force cosine similarity over the rows in scope, where scope is the per-user or per-agent namespace. There is deliberately no approximate-nearest-neighbor index; per-tenant row counts make exact search affordable (measured on one workstation: p50 latency 17 ms at 1,000 memories, 191 ms at 10,000, about 1 s at 50,000), and "no index to update" is itself a simplification. An optional local cross-encoder reranker can rescore candidates for harder queries. Section 5.4 shows where it helps and where it does not.

Bi-temporal layer (optional). For workloads that ask what was true at a particular time, GENOME can additionally maintain a belief store that records each fact with its domain-time validity, meaning the time the fact became true in the world, which is not the time it was ingested. Facts arriving out of chronological order therefore land in the right place on the timeline, and a query `facts_valid_at(entity, T)` is answered from structure rather than from prose. Unlike the core write path, this layer does run a structured extraction step at ingest; its output is a dated event record that can be audited mechanically against the source conversation, and Section 5.5 reports that audit alongside the accuracy numbers.

4 Experimental setup

Every comparison in this paper runs inside one harness. All systems answer with the same responder model (Claude Haiku 4.5 unless a section states otherwise), are graded by the same judge using Mem0's own published judge prompt at temperature 0, embed with the same encoder, and retrieve the same top-k. Only the memory layer changes. The shared encoder for the LoCoMo harness is OpenAI's `text-embedding-3-small`, for every system including GENOME: holding the encoder constant isolates the variable under test, which is LLM extraction at write time, not embedding quality. That choice deserves an explicit consequence: the headline tables measure GENOME in a hosted-encoder configuration, not the local default described in Section 3. The zero-network write property is verified separately against the local default, and the n = 205 LongMemEval run in Section 5.4, which used a local encoder for both systems, reaches the same parity conclusion in the fully local setting. Accuracy significance is a paired McNemar test on per-question correctness, which is the right test when two systems answer the same questions.

Two caveats belong here rather than in the limitations section, because they qualify everything that follows. First, the responder and the judge are the same model family, which is standard practice for LoCoMo but carries a self-preference risk; the risk applies equally to every system in the comparison, so it should not bias the between-system deltas, but absolute accuracies should be read with it in mind. Second, the LoCoMo numbers are single-run at a fixed seed without confidence intervals, so sub-percent gaps should be treated as noise, and I do treat them that way.

5 Results

5.1 In-window answer accuracy: parity (LoCoMo)

System	Headline	multi-hop	temporal	open-domain	single-hop
full-context (upper bound)	0.863	0.883	0.748	0.531	0.938
Memo	0.855	0.890	0.863	0.479	0.882
GENOME	**0.851**	0.851	0.798	0.500	0.911

Table 1. LoCoMo answer accuracy over the 1,540 headline questions (categories 1-4). Paired McNemar: GENOME vs Memo $\Delta = -0.004$, $p = 0.747$; GENOME vs full-context $\Delta = -0.012$, $p = 0.232$.

When the whole conversation fits in the context window, LoCoMo accuracy has a ceiling around 0.85 to 0.86 that every serious system reaches. GENOME reaches it. Its point estimate is nominally the lowest of the three, the differences are not statistically significant, and the honest statement is "no confirmed difference." I do not claim an in-window accuracy win, and I would flag that in-window LoCoMo is close to saturated and is not where a memory layer differentiates. What the table establishes is narrower and more useful: removing the LLM from the write path did not cost measurable accuracy.

5.2 When history overflows the window, retrieval wins decisively

The situation that justifies a memory layer is the one where full context is impossible. I built a 284,000-token history, past the 200,000-token window of the responder, and asked the same 120 questions. To avoid a strawman baseline, full-context was truncated from both ends separately: a prefix baseline keeps the first B tokens, and a recency baseline keeps the last B tokens, which is what a real chat application does.

System	Accuracy	Context tokens / query
GENOME (retrieval)	**0.817**	1,602
full-context, recency @128k	0.408	128,000
full-context, prefix @128k	0.300	128,000
full-context @32k (either end)	0.125-0.133	32,000

full-context @8k (either end)	0.067-0.117	8,000

Table 2. Accuracy on a 284k-token history that exceeds the responder's window. GENOME beats the stronger truncation by +0.409 while sending 80× less context per query.

Neither truncation direction rescues the baseline, because with 284k tokens of history no fixed window holds the evidence; the model simply cannot see out-of-window facts, and retrieval puts them back. The per-category gaps against the recency baseline are large across the board: single-hop 0.871 versus 0.500, multi-hop 0.810 versus 0.429, temporal 0.783 versus 0.217. Mem0 does not appear in this table by design: the comparison isolates a mechanism, retrieval versus a truncated window, rather than re-running the system comparison of Sections 5.1 and 5.4, and any retrieval-based memory should clear this bar. Running Mem0 over the same overflow set is straightforward future work.

5.3 Ingestion cost: zero LLM calls is an arithmetic fact

I instrumented Mem0's LLM client and GENOME's write path over an identical 80-turn conversation slice, with Mem0 configured to use the same Haiku 4.5 model as everywhere else in the harness.

System	LLM calls / message	Tokens (80 turns)	Wall-clock	Cost (80 turns)
Mem0	1.00	687,192 in + 4,940 out	164.4 s	$0.7119
GENOME	**0**	4,221 (embedding only)	19.2 s	**$0.00008**

Table 3. Measured ingestion over the same 80 turns. The 687k input tokens are Mem0's own extraction prompts.

The robust fact here is the call count, 0 versus 1.00 per message, because no pricing assumption changes it. The dollar multiplier does depend on which model you price: at Haiku pricing the full 5,882-turn corpus costs about $52 to ingest with Mem0 and about $0.006 with GENOME. GENOME's dollar figure prices the shared hosted encoder of Section 4, and its 19.2 s wall-clock, about 240 ms per message, is dominated by hosted-encoder round trips; with the default local encoder the API cost of a write is zero and the write itself takes roughly 10 ms. Projecting to a deployment of 10,000 users writing 50 messages a day, Mem0's yearly ingest bill runs from $159,000 with the cheapest hosted extraction model to $1.6M at Haiku pricing, against roughly $190 for GENOME, a factor of 837 to 8,433. One mechanism I did not measure but expect makes this an underestimate: Mem0 ships existing memories back to the LLM on each write, so its per-message cost should grow as the store fills. And the reason this is a win rather than a tradeoff is Section 5.1: the spend buys no measured accuracy.

5.4 A harder benchmark: LongMemEval, two runs, same conclusion

LoCoMo's saturation means retrieval improvements cannot show up in its answer accuracy, so I also ran LongMemEval-S, where each question comes with an average of 51 sessions of material and distractors. On pure retrieval, the local cross-encoder reranker concentrates its value exactly where retrieval is hard: multi-session hit@10 rises from 0.610 to 0.832, while easy single-session queries stay flat or dip slightly.

On answer accuracy I ran two independent head-to-heads. The first, 90 questions with a Sonnet responder to keep the answering step from bottlenecking: GENOME with rerank 0.700, GENOME dense 0.689, Mem0 0.622. GENOME leads nominally by +0.078, and the paired test says not significant (GENOME-only correct 14 versus Mem0-only 7, p = 0.19). The second run, 205 questions on a different backend with a local embedder for both systems: GENOME dense 0.595, Mem0 0.537, again nominally ahead, again not significant (34 versus 22, p = 0.14). Two runs, two backends, the same reading: GENOME is directionally ahead of Mem0 on the harder benchmark and never crosses the significance line. The honest conclusion is parity, and I note it would have been easy to publish the n = 90 table alone and let the +0.078 imply more than it proves.

The second run also produced a caveat worth keeping: with the weaker local embedder, reranking improved retrieval hit@10 (0.895 versus 0.827) but not answer accuracy (0.546 versus 0.595 dense). The reranker's answer-level benefit is embedder-dependent, and I report it as such rather than as a universal feature.

5.5 Point-in-time queries: where structure beats everything (with scope)

The results so far are about retrieval. The bi-temporal layer targets a different question entirely: not "what did the user say about X" but "what was true of X at time T," asked after the fact has changed several times. To measure it I built TempBelief, a deterministic, gold-validated benchmark in which entities' attributes change across 6 to 8 sessions, about a third of the updates are narrated out of chronological order, and distractors such as tentative plans must not change belief. All systems ingest the same raw natural-language turns; Mem0 runs at its strongest configuration (`infer=True` with history traversal). Six conversations, 108 point-in-time questions.

Split	GENOME belief	Mem0	GENOME dense	full-context
as-of (point-in-time)	**0.870**	0.676	0.407	0.139
current-value	0.833	0.861	0.972	1.000

Table 4. TempBelief, n = 108 as-of questions. Belief vs Mem0 on as-of: p = 0.0007 (28 belief-only vs 7 Mem0-only correct). Belief vs dense and vs full-context: p < 1e-5.

This is the one place in the paper where GENOME beats the baselines with statistical significance, and the mechanism is easy to state: overwrite-based memory destroys older values, and both raw retrieval and full context return the latest mention or fail to disambiguate dates in a cluttered window. Only the structure answers "as of March" reliably. To rule out judge leniency, a mechanical

extraction audit checks the belief store against the gold events: precision 0.972, recall 0.963, and every captured fact dated correctly. The accuracy reflects a correct store, not a generous grader.

The scope limits are just as important. On trivial current-value lookups the belief layer is the worst of the four systems (0.833 against full-context's 1.000), because extraction is noisier than simply reading the last mention; structure pays off only when time matters. And an early version of this experiment failed on relative-dated language ("8 months ago"): the audit traced the failure to a date-resolution bug in my ingest step, with domain-time accuracy at 0.323. After the fix, domain-time accuracy reached 0.829 with no regression on explicit dates, and a re-measured head-to-head on resolvable relative phrasing gave belief 0.741 versus Mem0 0.574 at n = 54, p = 0.137, a lead that trends but does not reach significance at that sample size. Genuinely vague dates ("about two years ago") remain hard for every system tested.

5.6 Negative results

Three ablations came out against the fashionable option, and they are worth as much as the wins.

Hybrid retrieval lost to plain dense. Using LoCoMo's annotated evidence turns as ground truth, with no LLM involved so retrieval is isolated: dense hit@10 is 0.715, dense with a recency rerank 0.708, and hybrid BM25-plus-dense 0.564. Keyword blending pulls in lexically similar but wrong turns. Graph-based retrieval is omitted rather than refuted, because the offline store never built an entity graph and the mode correctly fell back to dense; I make no claim either way, though an earlier full-pipeline graph variant scored far worse on answers (0.494), so the burden of proof sits with graph.

Summarizing old memories was a wash. Consolidation, meaning summarize old memories instead of deleting them, tested at strictly equal token budget on both sides: 0.260 versus 0.255, p = 0.861. Per-category direction hints that summaries help cross-session reasoning and hurt exact-fact lookup, which matches the mechanism, since a summary blurs the one verbatim detail a single-hop question needs. An earlier run had shown a large synthesis win; it turned out to be a token confound, with the summary arm fed far more context, and disappeared under an equal budget. I report the corrected number.

Reranking is not free accuracy. As Section 5.4 showed, its answer-level benefit appeared with a strong embedder and vanished with a weak one, despite improving retrieval in both cases.

6 Limitations

The evaluation was designed, run, and cross-checked by one person with automated tooling, and it has not yet been independently replicated; the entire harness ships in the repository precisely so that someone else can. The responder and judge share a model family, which is standard for these benchmarks but is a known bias risk. LoCoMo results are single-seed without confidence intervals. TempBelief is a benchmark I built myself; it is deterministic and gold-validated, and its

construction is in the repository for inspection, but self-built benchmarks deserve extra suspicion and I invite it. The LongMemEval samples (n = 90 and n = 205) are moderate, and the two runs used different backends, so their absolute numbers are not directly comparable; I use them only as two independent parity observations. Brute-force retrieval is a deliberate simplicity choice that becomes a real cost somewhere past 50,000 memories per tenant (about 1 s per query on my hardware); the escape hatch is Postgres with pgvector behind the same API, which I have not benchmarked here. Finally, GENOME's write path stores what the user said rather than an abstracted fact; workloads that need write-time abstraction, whatever those turn out to be, are outside what this evaluation can speak to.

7 Reproducing these results

Each finding regenerates from one script. The LoCoMo and LongMemEval datasets are not redistributed here (LoCoMo is CC BY-NC 4.0); `benchmarks/data/README.md` documents how to obtain them.

```
git clone https://github.com/NORTHTEKDevs/genome
cd genome
pip install -e .                          # the library alone is: pip install genome-memory
python benchmarks/verdict.py              # Table 1: in-window accuracy + McNemar
python benchmarks/haystack_report.py      # Table 2: overflow stress test
python benchmarks/ingest_cost.py --n 80 # Table 3: measured ingestion cost
python benchmarks/lme_qa.py --n 90        # Section 5.4: LongMemEval head-to-head
python benchmarks/tempbelief_run.py       # Table 4: point-in-time benchmark
python benchmarks/retrieval_quality.py  # Section 5.6: retrieval-mode ablation
python benchmarks/budget_fair.py          # Section 5.6: token-fair synthesis test
python -m genome.verify                   # offline write-path receipt, no API key
```

The last command needs no API key at all: it writes memories with all outbound sockets blocked and prints a pass/fail receipt for the offline, zero-LLM, roughly-10 ms write path of the local default configuration on your own machine.

8 Availability

GENOME is Apache-2.0 at `github.com/NORTHTEKDevs/genome`, on PyPI as `genome-memory`, and in the official Model Context Protocol registry as `io.github.NORTHTEKDevs/genome`, where it runs as a fully local MCP memory server for any MCP-capable agent. Raw run artifacts for any specific number are available on request, since the dataset licenses prevent shipping them in the repository.

References

[1] P. Chhikara, D. Khant, S. Aryan, T. Singh, D. Yadav. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413, 2025.

[2] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, J. E. Gonzalez. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560, 2023.

[3] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, Y. Fang. Evaluating Very Long-Term Conversational Memory of LLM Agents. arXiv:2402.17753, 2024.

[4] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, D. Yu. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. arXiv:2410.10813, 2024.

[5] P. Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv:2005.11401, 2020.

[6] N. Reimers, I. Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. arXiv:1908.10084, 2019.

Independent replication of any result in this paper is welcome and encouraged; the harness is public, and I will send raw run artifacts to anyone who asks. Contact: info@northtek.io.